Contents

1 [emoji] 桌面小组件–开发者文档 1
1.1 目录 1
1.2 项目结构 2
1.3 架构概览 2
1.3.1 关键设计原则 2
1.4 Tile Registry –磁贴注册表 2
1.4.1 `registerTile(descriptor)` 参数说明 3
1.4.2 `buildDefaultPos(headerH)` –初始位置生成器 3
1.5 数据层–DataContext 3
1.5.1 `settings` 对象结构 4
1.5.2 内置 Hooks 4
1.6 CSS 设计系统 4
1.6.1 CSS 变量（在 `.R` 上定义） 4
1.6.2 常用 CSS 类 5
1.7 新增磁贴完整教程 5
1.7.1 Step 1：在 DataProvider 中添加数据（可选） 5
1.7.2 Step 2：编写组件 6
1.7.3 Step 3：注册磁贴 6
1.7.4 Step 4：完成！ 7
1.8 localStorage 键表 7
1.9 开发命令 7
1.9.1 课程表转换工具 8
1.10 常见问题 8

1 [emoji] 桌面小组件–开发者文档

> 本文档面向希望在此项目基础上新增磁贴、修改样式或理解内部架构的开发者。

1.1 目录

1. 项目结构
2. 架构概览
3. Tile Registry –磁贴注册表
4. 数据层–DataContext
5. CSS 设计系统
6. 新增磁贴完整教程
7. localStorage 键表
8. 开发命令

1.2 项目结构

```
forClaudeDispatch/
├── desktop-widget.jsx     # 主文件: 所有 React 组件 + CSS (all-in-one)
├── main.jsx               # React 入口, 挂载 <App/>
├── index.html             # HTML 壳
├── vite.config.js         # Vite 配置 (dev port: 15173)
├── schedule.json          # 个人课程表数据 (gitignored)
├── schedule.json.sample   # 课程表格式示例
├── doc_to_schedule_json.py # Word XML → schedule.json 转换工具
├── src-tauri/
│   ├── tauri.conf.json    # 窗口配置 (透明、无边框、always_on_top等)
│   ├── capabilities/default.json  # Tauri 权限配置
│   └── src/
│       ├── main.rs
│       └── lib.rs         # Tauri 后端入口 (目前为空)
└── CONTRIBUTING.md        # 本文档
```

1.3 架构概览

```
App
└── ThemeProvider          # 全局深/浅色模式 Context
    └── DataProvider      # 全局数据 Context (课程、DDL、笔记等所有状态)
        ├── <style>        # 所有 CSS (内联 template literal)
        └── Layout
            ├── Dragbar    # 顶部窗口拖动条 (data-tauri-drag-region)
            ├── TopBar     # 时钟 / 天气 / 一言 / 主题切换
            ├── Summary    # 今日课程数 / DDL 数快览
            ├── FreeCanvas # 磁贴画布 (position:absolute 自由布局)
            │   ├── TileA (fc 容器 + fc__handle + fc__rsz)
            │   ├── TileB
            │   └── ...   # 从 TILE_REGISTRY 自动渲染
            └── StickyNotes # 浮动便签 (独立于磁贴系统)
```

1.3.1 关键设计原则

- **Single-file**: 所有逻辑和样式都在 `desktop-widget.jsx`，便于 Tauri 单文件部署。
- **CSS-in-JSX**: CSS 写在 `<style>{\…'}'` 中，通过 CSS 变量实现主题切换。
- **Dom-direct drag**: 拖拽时直接操作 `el.style`，不触发 React re-render，确保流畅。
- **Persist everything**: 所有用户状态通过 `usePersistedState` 自动写入 `localStorage`。

1.4 Tile Registry –磁贴注册表

注册表是一个全局 Map，是定义哪些磁贴存在的**唯一入口**。

```
const TILE_REGISTRY = new Map(); // id → TileDescriptor
```

1.4.1 registerTile(descriptor) 参数说明

参数	类型	必填	说明
id	string	□	全局唯一 key，用于 localStorage 位置存储
label	string	□	把手上显示的中文名称
icon	string	□	把手上显示的 emoji
component	React component	□	磁贴的渲染函数
defaultW	number	□	默认宽度（px），默认 300
defaultH	number | undefined	□	默认高度，undefined = 随内容自动撑开
defaultPos	{ col: 0-3, row: 0+ }	□	初始网格坐标。col: 0~3 列（间距 316px），row：行（间距 340px）

1.4.2 buildDefaultPos(headerH) –初始位置生成器

在 Layout 内部调用，根据注册表数据计算每个磁贴的初始像素坐标：

```
col → x:  [16, 332, 648, 964]  (4列)
row → y:  headerH + 16 + row * 340
```

新注册的磁贴如果还没有 localStorage 记录，会自动使用此处的计算结果。

1.5 数据层–DataContext

所有组件通过 useData() hook 获取共享状态：

```
const {
  // 课程相关
  courses, setCourses,        // 今日课程列表 [{ time, name, loc, c }]
  weekData, setWeekData,      // 全周数据 { 1:[...], 2:[...], ..., 7:[...] }
  settings, setSettings,      // 全局设置对象（见下）

  // 任务相关
  ddl, setDdl,                // DDL 列表 [{ id, title, due, tag }]
  todo, setTodo,              // 待办列表 [{ id, text, done }]
  countdown, setCountdown,    // 自定义倒计时 [{ id, label, date, emoji }]
```

```
  // 便签
  notes, setNotes,           // 便签 [{ id, x, y, text, color }]
} = useData();
```

1.5.1 settings 对象结构

```
{
  weather: { lat, lon, city },  // 天气位置配置
  semesterStart: "2026-03-02",  // 开学日期（周一），用于计算周次
  pos: {                        // 各磁贴位置（由 Layout 自动维护）
    CourseId: { x, y, w, h },
    ...
  }
}
```

1.5.2 内置 Hooks

Hook	返回	说明
useTheme()	{ dark, toggle }	深/浅色模式状态和切换函数
useData()	见上表	全局数据 Context
useClock()	Date	每秒更新的当前时间
useHitokoto()	{ text, from, refresh }	一言 API
useTodayCourses()	Course[]	今日课程（已做周次过滤）
useWeekCourses(day)	Course[]	指定星期几的本周课程
usePersistedState(key, init)	[state, setState]	自动读写 localStorage

---

1.6 CSS 设计系统

所有样式在 App 的 <style> JSX 标签内定义。通过 CSS 变量实现主题切换:

1.6.1 CSS 变量（在 .R 上定义）

变量	Light	Dark	用途
--bg	rgba(244,243,239,.7)	rgba(25,25,24,.7)	窗口背景
--card	rgba(255,255,255,.75)	rgba(36,36,35,.75)	磁贴/卡片背景
--bd	rgba(230,229,224,.5)	rgba(255,255,255,.08)	边框
--bd2	rgba(216,215,210,.6)	rgba(255,255,255,.15)	悬停边框
--tx	#1c1c1a	#e8e8e4	主文字
--tx2	#5c5c58	#a0a098	次要文字
--tx3	#9c9c96	#686864	辅助文字/占位
--ac	#4a7fd8	#6a9de8	强调色（蓝）

变量	Light	Dark	用途
--red	#d14	#e55	警告/紧急
--orange	#d48a3b	#e8a050	次警告
--green	#3ba868	#50c878	成功/正常
--f	'LXGW WenKai Screen'	同	正文字体
--fm	'LXGW WenKai Mono'	同	等宽/数字字体
--r	12px	同	卡片圆角
--rs	8px	同	小元素圆角
--shadow	…	…	默认阴影
--shadow-hover	…	…	悬停阴影

1.6.2 常用 CSS 类

类	含义
.p	磁贴面板容器（card background + border + shadow）
.ph	面板标题行（flex, space-between）
.ph h2	面板标题文字
.ph__r	面板标题右侧操作区
.gb	ghost button（边框透明小按钮）
.sb	solid button（填色按钮）
.rm	remove button（删除按钮，红色悬停）
.fm	表单行容器（flex + gap）
.cnt	计数/说明文字（等宽小字）
.fc	磁贴浮动容器（position:absolute）
.fc__handle	拖动把手（顶部条纹）
.fc__rsz	调整大小把手（右下角三角）

1.7 新增磁贴完整教程

以下以新增一个「习惯打卡 (HabitTracker)」磁贴为例：

1.7.1 Step 1：在 DataProvider 中添加数据（可选）

如果你的磁贴需要持久化自己的数据，在 DataProvider 内添加：

```
const [habits, setHabits] = usePersistedState("widget_habits", [
  { id: 1, name: " 早起", icon: "[emoji]", streak: 0 },
  { id: 2, name: " 运动", icon: "[emoji]", streak: 3 },
]);
// 在 DataCtx.Provider value 中加上 habits, setHabits
```

1.7.2 Step 2：编写组件

```
function HabitTracker() {
  const { habits, setHabits } = useData();
  const today = new Date().toDateString();

  const check = (id) => {
    setHabits(prev => prev.map(h =>
      h.id === id
        ? { ...h, streak: h.streak + 1, lastChecked: today }
        : h
    ));
  };

  return (
    <div className="p">
      <div className="ph">
        <h2>习惯打卡</h2>
        <span className="cnt">{habits.filter(h => h.lastChecked ===
        ↪  today).length}/{habits.length} 已完成</span>
      </div>
      <div>
        {habits.map(h => (
          <div key={h.id} style={{ display:"flex", alignItems:"center",
          ↪  gap:10, padding:"6px 0" }}>
            <button className="gb" onClick={() => check(h.id)}
              style={{ fontSize:18, padding:"2px 6px" }}>
              {h.lastChecked === today ? "□" : h.icon}
            </button>
            <div>
              <div style={{ fontWeight:500 }}>{h.name}</div>
              <div className="cnt">连续 {h.streak} 天</div>
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

1.7.3 Step 3：注册磁贴

> 紧接在组件定义结束后调用 `registerTile`，**不需要修改 Layout 或其他任何代码。**

```
registerTile({
  id:          "HabitTracker",
  label:       " 习惯打卡",
```

```
  icon:        "[emoji]",
  component:   HabitTracker,
  defaultW:    260,            // 稍窄一些
  defaultPos:  { col: 3, row: 0 },  // 放在第 4 列第 1 行
});
```

1.7.4 Step 4：完成！

刷新页面，新磁贴会自动出现在指定位置，可拖拽、可缩放、位置自动记忆。

1.8 localStorage 键表

键名	内容
widget_dark	"true" / "false" 深色模式状态
widget_courses	今日课程数组（mock 数据）
widget_week	全周课程 Map
widget_ddl	DDL 条目数组
widget_todo	待办条目数组
widget_countdowns	自定义倒计时数组
widget_notes	便签数组（包含位置信息）
widget_settings	全局设置（含所有磁贴位置 pos）

> **重置布局**:在浏览器控制台执行 `localStorage.removeItem('widget_settings')` 后刷新，所有磁贴回到默认位置。

1.9 开发命令

```
# 安装依赖
npm install

# 启动开发服务器（端口 15173，支持热更新）
npm run dev

# 在 Tauri 窗口中启动（需要 Rust 环境）
npx tauri dev

# 构建成 .exe 安装包
npx tauri build
# 输出路径: src-tauri/target/release/bundle/
```

1.9.1 课程表转换工具

```
# 将教务系统导出的 Word 课程表转换为 schedule.json
python doc_to_schedule_json.py " 你的课程表.doc"
python doc_to_schedule_json.py " 课程表.doc" my_schedule.json
python doc_to_schedule_json.py " 课程表.doc" --stdout   # 打印到终端
```

1.10 常见问题

Q: 磁贴消失了怎么办?
A: 打开控制台执行 `localStorage.removeItem('widget_settings')` 然后刷新。

Q: 磁贴拉到顶部被 Header 遮住了?
A: 不会。拖拽时 Y 轴下限被强制限制在 `HEADER_H` (130px)。如果旧数据导致位置异常，清空 `widget_settings` 即可。

Q: 如何调整 Header 高度约束?
A: 修改 `Layout` 函数顶部的 `const HEADER_H = 130`。

Q: 如何给磁贴指定固定高度?
A: 在 `registerTile` 中设置 `defaultH: 400`，或用户直接拖动右下角调整大小手柄。

Q: 如何让某个磁贴不可调整大小?
A: 目前不支持，如有需要可在 `Layout` 的 JSX 中根据 `tile.id` 条件性地不渲染 `fc__rsz`。